JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class OptionalInt

java.lang.Object
    java.util.OptionalInt

```
public final class OptionalInt
extends Object
```

A container object which may or may not contain a `int` value. If a value is present, `isPresent()` will return `true` and `getAsInt()` will return the value.

Additional methods that depend on the presence or absence of a contained value are provided, such as `orElse()` (return a default value if value not present) and `ifPresent()` (execute a block of code if the value is present).

This is a value-based class; use of identity-sensitive operations (including reference equality (==), identity hash code, or synchronization) on instances of `OptionalInt` may have unpredictable results and should be avoided.

Since:

1.8

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static OptionalInt	empty()	Returns an empty OptionalInt inst
boolean	equals(Object obj)	

		Indicates whethe other object is "e this OptionalInt.
int	**getAsInt**()	If a value is presen OptionalInt, ret value, otherwise NoSuchElementE
int	**hashCode**()	Returns the hash value of the pres if any, or 0 (zero) value is present.
void	**ifPresent**(**IntConsumer** consumer)	Have the specifie consumer accept value if a value is otherwise do not
boolean	**isPresent**()	Return true if th value present, ot false.
static **OptionalInt**	**of**(int value)	Return an Optio with the specifie present.
int	**orElse**(int other)	Return the value present, otherwis other.
int	**orElseGet**(**IntSupplier** other)	Return the value present, otherwis other and return result of that inv
<X extends **Throwable**> int	**orElseThrow**(**Supplier**<X> exceptionSupplier)	Return the conta value, if present, otherwise throw exception to be c the provided sup
String	**toString**()	Returns a string representation o object.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Detail

empty

```
public static OptionalInt empty()
```

Returns an empty `OptionalInt` instance. No value is present for this OptionalInt.

API Note:

Though it may be tempting to do so, avoid testing if an object is empty by comparing with == against instances returned by Option.empty(). There is no guarantee that it is a singleton. Instead, use `isPresent()`.

Returns:

an empty OptionalInt

of

```
public static OptionalInt of(int value)
```

Return an `OptionalInt` with the specified value present.

Parameters:

value - the value to be present

Returns:

an OptionalInt with the value present

getAsInt

```
public int getAsInt()
```

If a value is present in this `OptionalInt`,
returns the value, otherwise throws `NoSuchElementException`.

Returns:

the value held by this OptionalInt

Throws:

NoSuchElementException - if there is no value present

See Also:

`isPresent()`

isPresent

```
public boolean isPresent()
```

Return `true` if there is a value present, otherwise `false`.

Returns:

`true if there is a value present, otherwise false`

ifPresent

```
public void ifPresent(IntConsumer consumer)
```

Have the specified consumer accept the value if a value is present, otherwise do nothing.

Parameters:

`consumer - block to be executed if a value is present`

Throws:

`NullPointerException - if value is present and consumer is null`

orElse

```
public int orElse(int other)
```

Return the value if present, otherwise return `other`.

Parameters:

`other - the value to be returned if there is no value present`

Returns:

`the value, if present, otherwise other`

orElseGet

```
public int orElseGet(IntSupplier other)
```

Return the value if present, otherwise invoke `other` and return the result of that invocation.

Parameters:

other - a IntSupplier whose result is returned if no value is present

Returns:

the value if present otherwise the result of other.getAsInt()

Throws:

NullPointerException - if value is not present and other is null

orElseThrow

```
public <X extends Throwable> int orElseThrow(Supplier<X> exceptionSupplier)
                                        throws X
```

Return the contained value, if present, otherwise throw an exception to be created by the provided supplier.

API Note:

A method reference to the exception constructor with an empty argument list can be used as the supplier. For example, IllegalStateException::new

Type Parameters:

X - Type of the exception to be thrown

Parameters:

exceptionSupplier - The supplier which will return the exception to be thrown

Returns:

the present value

Throws:

X - if there is no value present

NullPointerException - if no value is present and exceptionSupplier is null

X extends Throwable

equals

```
public boolean equals(Object obj)
```

Indicates whether some other object is "equal to" this OptionalInt. The other object is considered equal if:

- it is also an OptionalInt and;
- both instances have no value present or;
- the present values are "equal to" each other via ==.

Overrides:

equals in class Object

Parameters:

obj - an object to be tested for equality

Returns:

{code true} if the other object is "equal to" this object otherwise false

See Also:

Object.hashCode(), HashMap

hashCode

public int hashCode()

Returns the hash code value of the present value, if any, or 0 (zero) if no value is present.

Overrides:

hashCode in class Object

Returns:

hash code value of the present value or 0 if no value is present

See Also:

Object.equals(java.lang.Object), System.identityHashCode(java.lang.Object)

toString

public String toString()

Returns a string representation of the object. In general, the `toString` method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read. It is recommended that all subclasses override this method.

The `toString` method for class `Object` returns a string consisting of the name of the class of which the object is an instance, the at-sign character `@', and the unsigned hexadecimal representation of the hash code of the object. In other words, this method returns a string equal to the value of:

```
getClass().getName() + '@' + Integer.toHexString(hashCode())
```

Returns a non-empty string representation of this object suitable for debugging. The exact presentation format is unspecified and may vary between implementations and versions.

Overrides:

`toString` in class `Object`

Implementation Requirements:

If a value is present the result must include its string representation in the result. Empty and present instances must be unambiguously differentiable.

Returns:

the string representation of this instance

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.